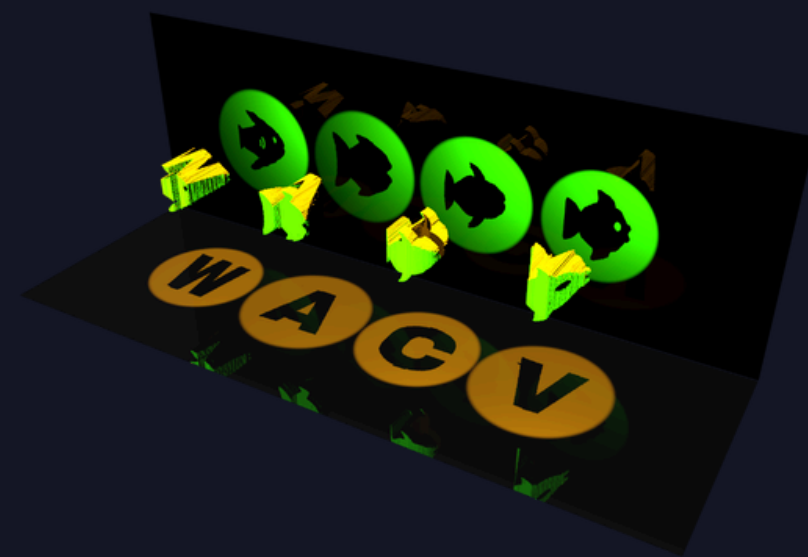


Esculturas que engañan a la luz

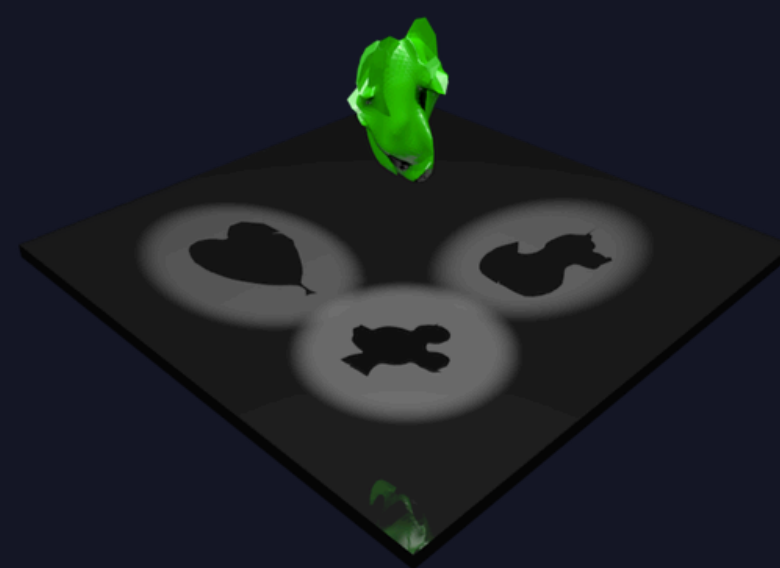
Replicación y evaluación de Shadow Art mediante renderizado diferenciable

Erick Lara · Nicolás Medel Correa · Vicente Retamal

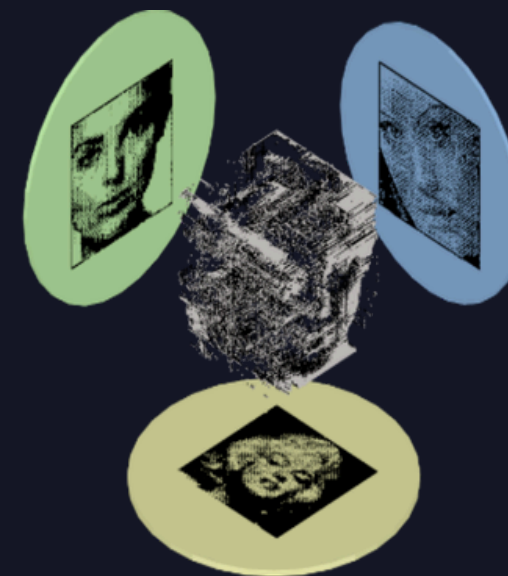
Basado en Sadekar et al., “Differentiable Shadow Art” — WACV 2022



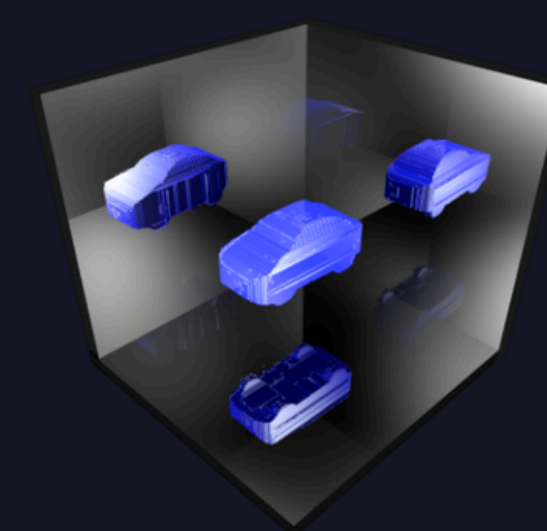
(a)



(b)



(c)

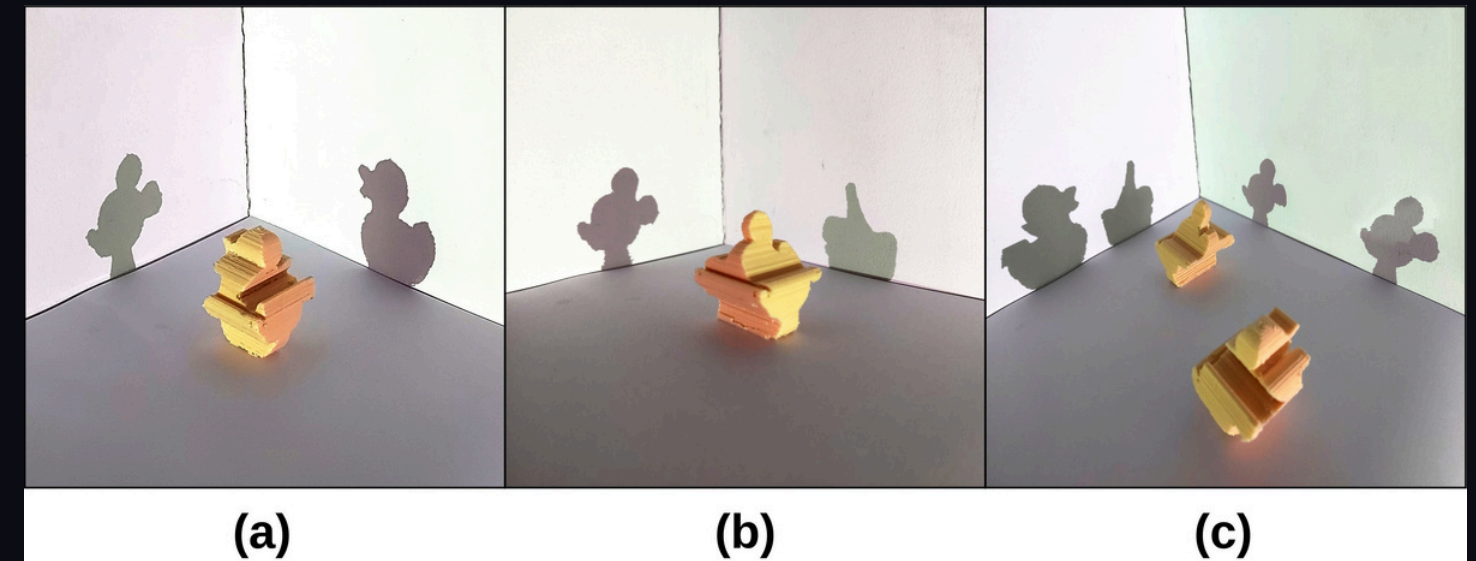


(d)



¿Qué es Shadow Art?

- Un único objeto 3D proyecta sombras distintas según la dirección de la luz.
- Cada sombra reproduce una silueta objetivo diferente.
- Es una escultura computacional que puede imprimirse en 3D.
- Nos motiva preguntar hasta dónde llega realmente el método.



Ejemplos de shadow art impreso en 3D

Pregunta de investigación

01 ¿Qué tan bien generaliza el método con siluetas de distinta complejidad geométrica?

02 ¿Cómo afecta la representación elegida: voxel frente a mesh?

03 ¿Cómo influye la cantidad de vistas usadas en la reconstrucción 3D?

Nuestra contribución

Replicamos el método del paper y **profundizamos en dos frentes: los datos y la evaluación.**

● Dataset propio

15 siluetas nuevas: 5 generadas con OpenCV + 10 de openclipart (CC0), clasificadas por compacidad en simple, media y alta.

● Métricas adicionales

Sumamos ROI Pixel Accuracy, Precision y Recall a las métricas del paper (IoU y Dice) para diagnosticar el tipo de falla.

● Experimentos espejo

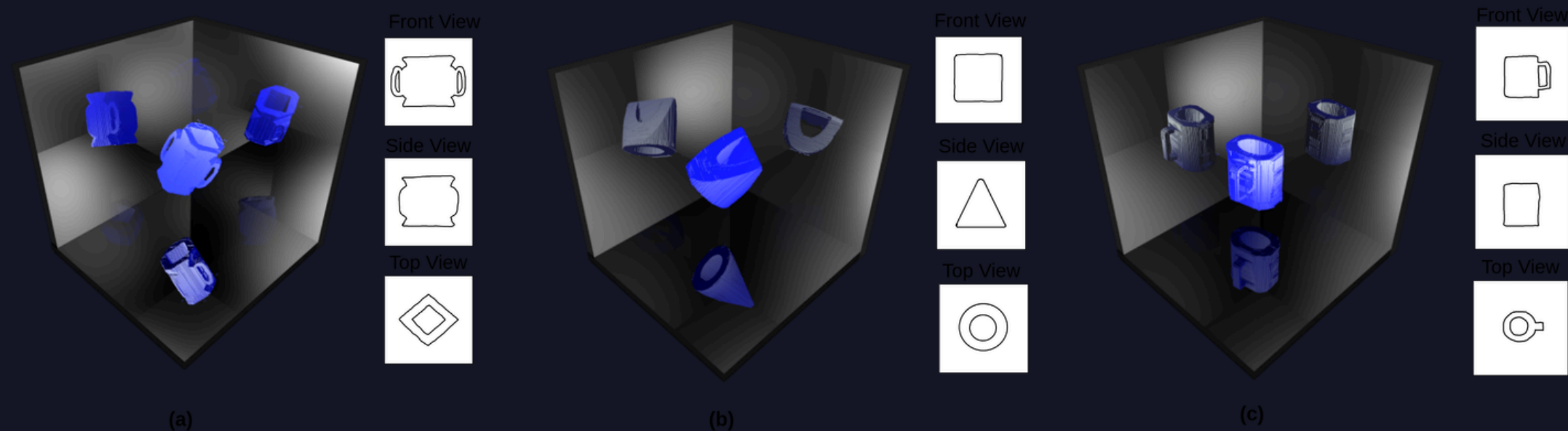
Recreamos la estructura de la Figura 4 usando solo nuestras siluetas, para medir la generalización del método.

● Reconstrucción 3D

Modelos car y rifle (Free3D, Blender) variando la representación y el número de vistas.

Dos datasets y reconstrucción 3D

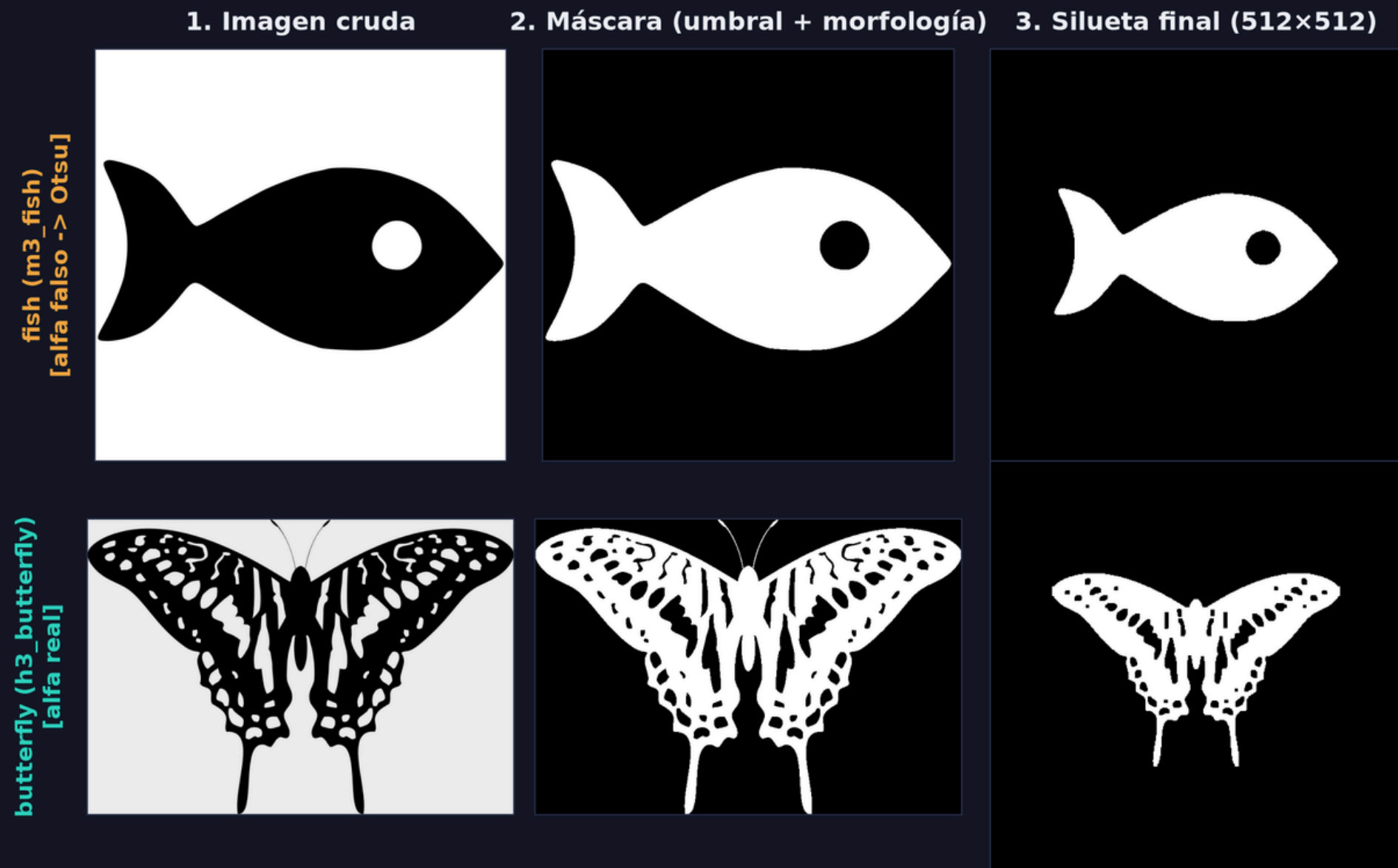
- Dataset del paper: 15 siluetas PNG binarias y usamos 9 como baseline de referencia.
- Dataset propio: 15 siluetas nuevas (10 de openclipart CC0 + 5 generadas con OpenCV).
- Modelos car y rifle desde Free3D, rasterizados en Blender para la reconstrucción 3D.
- Todas las siluetas: 512×512, figura blanca sobre fondo negro.



Objetos y siluetas usados en los experimentos

Segmentación de siluetas

Segmentación: de imagen cruda a silueta binaria

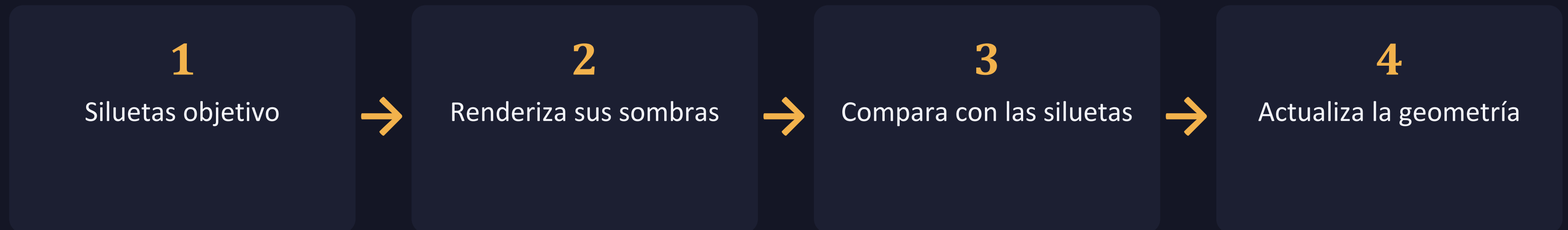


1. **Extracción de máscara:** Verifica si Alfa real si existe, si es "falso" (100% opaco), se usa Otsu sobre escala de grises.
2. **Limpieza morfológica:** Apertura + cierre (kernel 5×5) para eliminar ruido y artefactos.
3. **Recorte + padding:** Bounding box de la figura con 15% de margen.
4. **Normalización de tamaño:** Reescalado + centrado en canvas 512×512, binarizado a 0/255.

fish: PNG con canal alfa 100% opaco ("falso") → se ignora el alfa y se usa Otsu sobre escala de grises.
butterfly: PNG con transparencia real → se usa el canal alfa directamente, preservando los huecos finos de las alas.

Cómo funciona el pipeline

El punto de partida son las siluetas objetivo. La geometría inicial es un volumen (voxel) o una esfera (mesh) que se optimiza para reproducirlas.



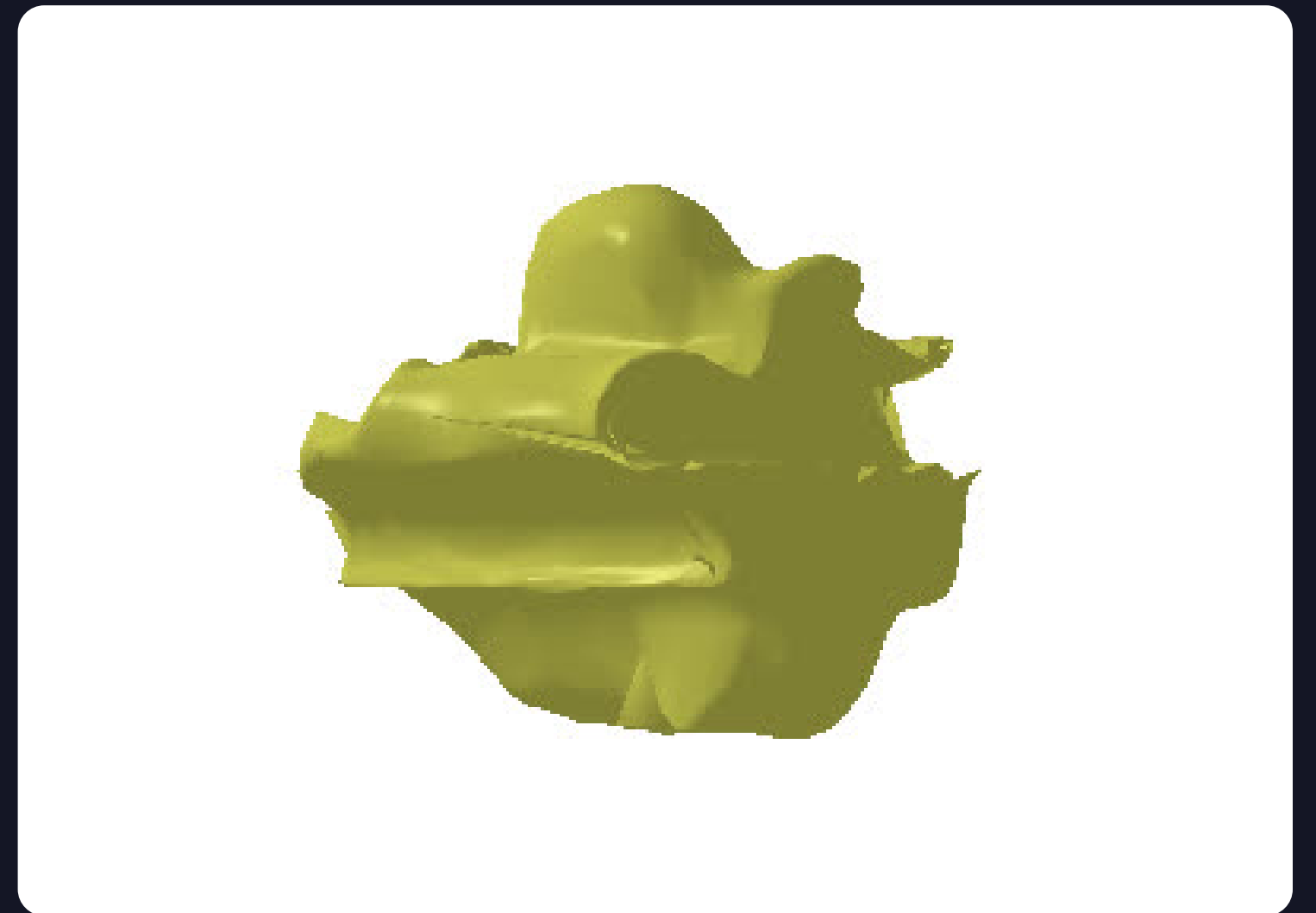
se repite en cada iteración hasta converger

Renderizado diferenciable + descenso de gradiente. Al converger: voxel se convierte a malla con cubify y mesh entrega la malla final.

Cómo funciona Mesh

Base aditiva

- Representación: una malla cerrada que parte de una esfera (ico_sphere).
- Se optimizan offsets por vértice, se renderiza la silueta desde cada vista.
- Pérdidas: silueta + L1 + regularizadores (edge, laplaciano, normales), optimizador SGD.
- Produce una superficie suave y cerrada (watertight)
- No cambia de topología: no puede perforar, así que añade material.



Optimización con representación mesh (animado)

Cómo funciona Voxel

Base sustractiva

- Representación: un volumen de densidades 128^3 (rejilla 3D de ocupación).
- Se renderiza el volumen desde cada vista y se compara con la silueta objetivo.
- Pérdidas: silueta + L1 (MS-SSIM opcional); optimizador Adam.
- Puede abrir agujeros y perforaciones libremente (topología flexible).
- Al converger se convierte a malla con cubify (umbral 0.05).



Optimización con representación voxel (animado)

Dataset propio: 15 siluetas por complejidad

Compacidad

$$C = 4\pi \cdot \text{Área} / P^2$$

Umbrales

Simple [0.42 – 1]

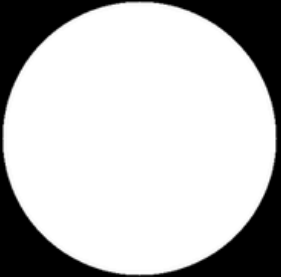
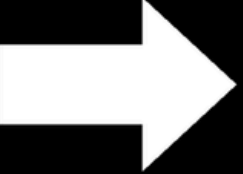
Media [0.23 – 0.42]

Alta [0.02 – 0.23]

Fuentes

5 generadas con OpenCV + 10 de openclipart (CC0).

Límite: aspect ratios extremos (guitarra 3.03) y áreas mínimas (≈4 %).

SIMPLE 5 siluetas					
	circulo C = 0.91	rectangulo C = 0.76	cruz C = 0.51	flecha C = 0.49	house C = 0.48
					
	bird C = 0.42	tree C = 0.39	fish C = 0.34	star C = 0.28	guitar C = 0.24
					
	sitting_cat C = 0.22	running_person C = 0.20	butterfly C = 0.11	snowflake C = 0.08	bike C = 0.02

Métricas de evaluación

Del paper

- IoU (intersección sobre unión)
- Dice

Agregamos

- ROI Pixel Accuracy
- Precision
- Recall

Precision penaliza la masa sobrante; **Recall** penaliza las omisiones. Juntas diagnostican el **tipo de falla**, no solo la similitud global.

¿Por qué construimos un baseline propio?

- Los valores de IoU y Dice no coinciden exactamente con la Figura 4 del paper.
- Causas probables: evolución de librerías (Python, NumPy, PyTorch3D) y ausencia de seeds.
- Decisión: usar nuestros propios resultados como baseline de comparación.
- Así comparamos siempre bajo el mismo hardware y software.



Reproducibilidad

Sin semillas fijas, los resultados no son replicables al pixel. El baseline propio hace justas las comparaciones.

Generalización: experimentos espejo

Recreamos la estructura de la Figura 4 usando solo nuestras siluetas.

Exp. 2a · media + alta

Generaliza casi perfecto

–8 %

caída de IoU (baseline)

Exp. 3b · 1 media + 2 alta

Se desploma

–21 %

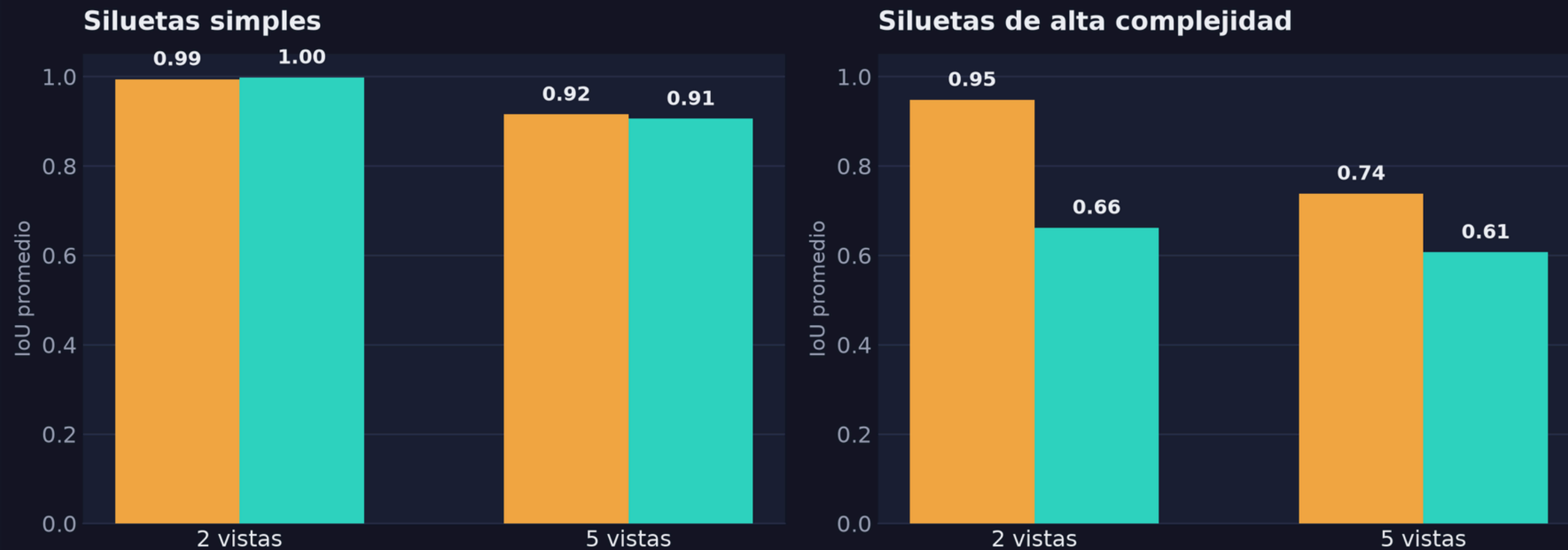
caída de IoU

Combinar siluetas geométricamente incompatibles rompe el método.

¿Cómo influye la cantidad de vistas?

¿Cómo influye la cantidad de vistas? (misma complejidad)

Voxel Mesh



IoU promedio del experimento (media entre todas las vistas). Mismas siluetas por nivel de complejidad, variando solo n° de vistas.

● RESULTADOS

¿Cómo influye la cantidad de vistas?

Panorama general: IoU promedio por cantidad de vistas



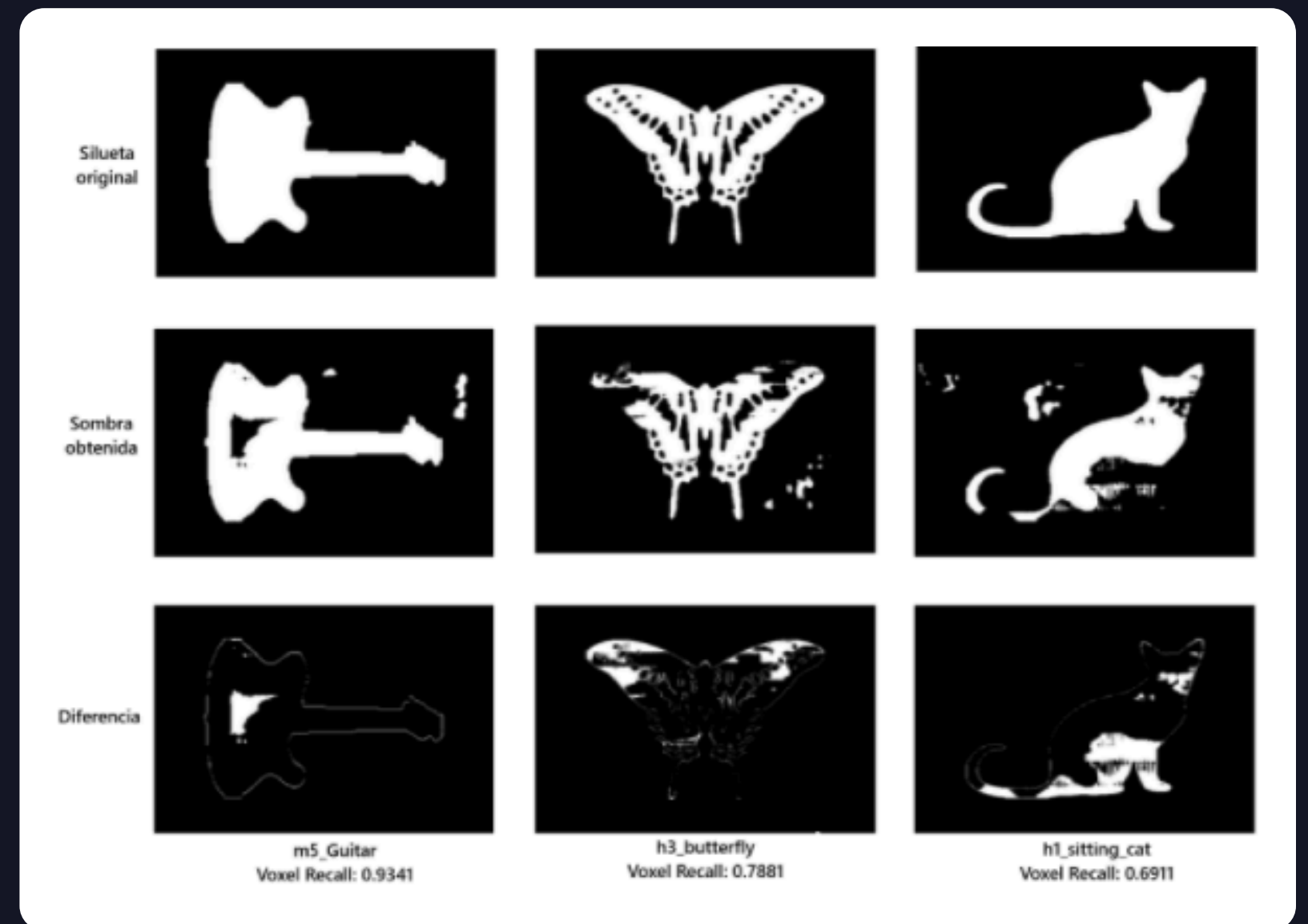
Promedio de todos los experimentos por bucket de vistas (composición de siluetas no controlada: los experimentos con más vistas incluyen más siluetas de alta complejidad, lo que confunde el efecto puro de n° de vistas).

● RESULTADOS

Cómo falla Voxel

Falla por sustracción destructiva

- Voxel talla y elimina bloques dentro de un volumen delimitado.
- Siluetas de área mínima y aspect ratio extremo (la guitarra) erosionan la masa central.
- Contraintuitivo: las siluetas más simples pueden perder más masa.



Silueta original · sombra obtenida · diferencia

● RESULTADOS

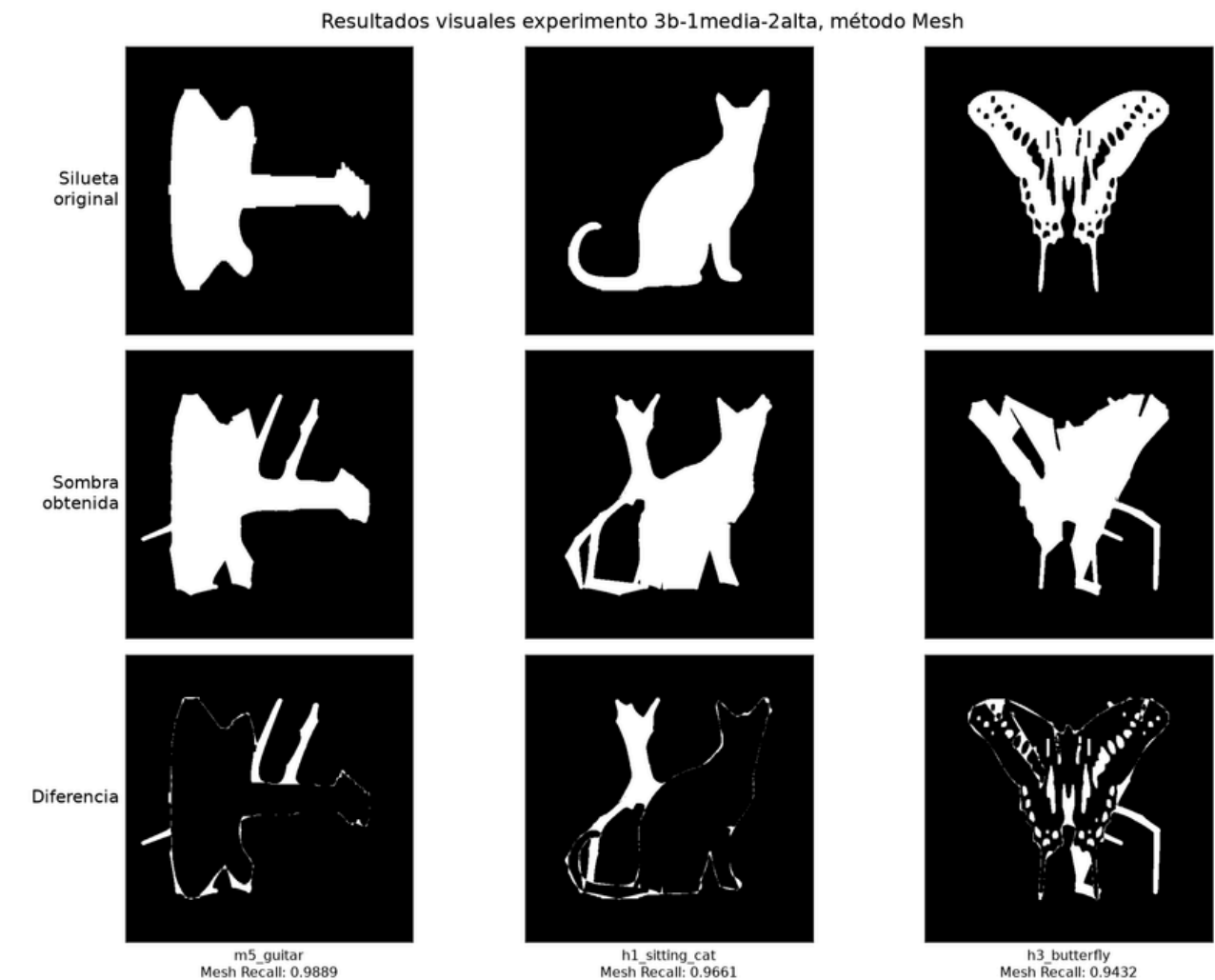
Cómo falla Mesh

Falla por adición de masa extra

- Mesh deforma una malla cerrada que no puede generar perforaciones.
- Construye “puentes” de material para alcanzar extremidades dispersas.
- El problema se traslada de Recall a Precision.

Voxel P 97.9 · R 91.8

Mesh P 90.4 · R 98.4

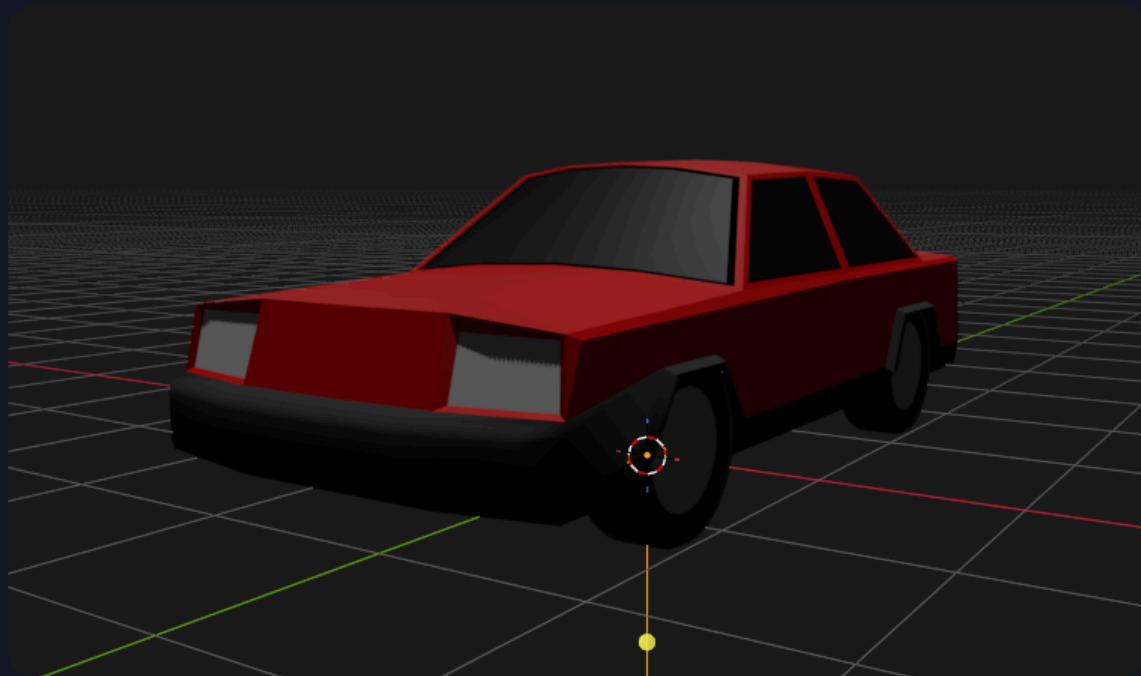


Resultado del experimento Mesh

● RESULTADOS

Reconstrucción 3D: Car y rifle

Car · Simple, simétrico

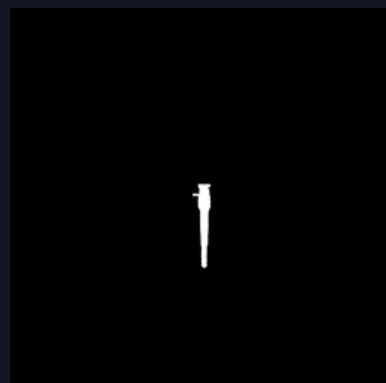
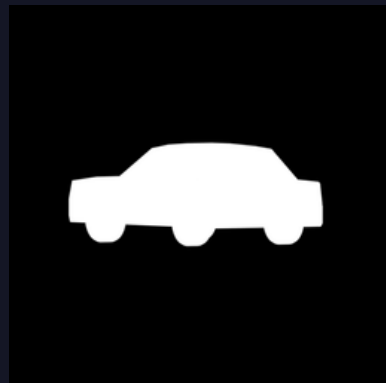
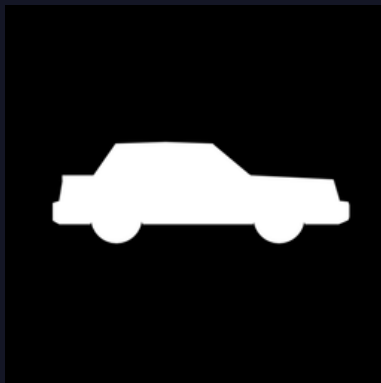
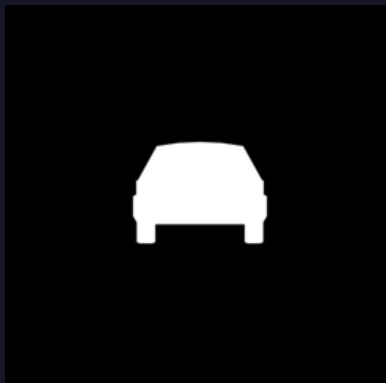


Rifle · Complejo, irregular



Descubrir que tan bien generaliza el modelo para reconstruir un modelo 3D a partir de sus siluetas.

Reconstrucción 3D: Car y rifle

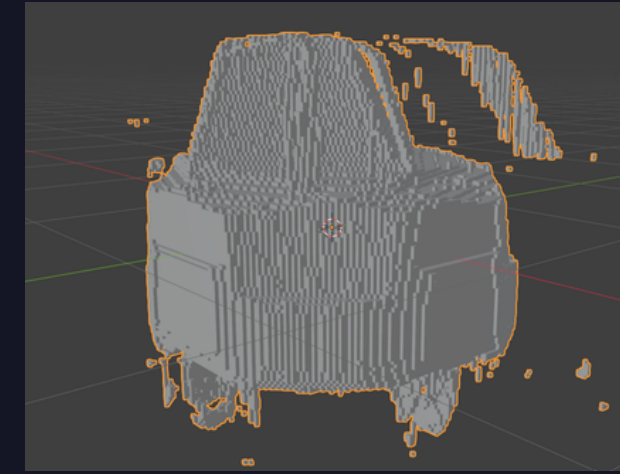
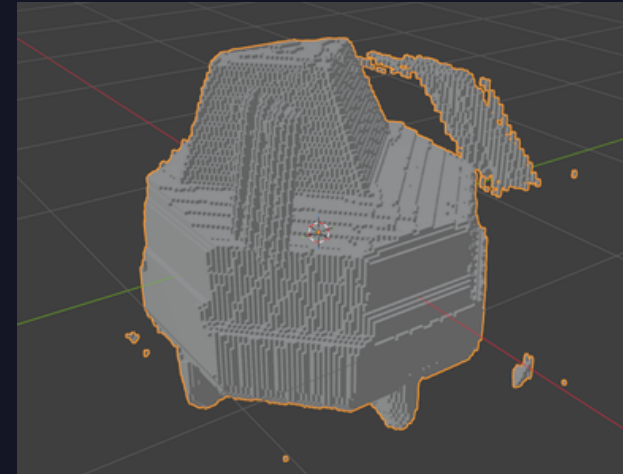


- Se obtuvieron siluetas de objetos 3D desde disintos ángulos mediante Blender.
- Por cada modelo se capturaron 8 siluetas correspondiente a cada uno de sus lados y puntos intermedios entre los mismos.
- Se comparó la calidad de las reconstrucciones de cada modelo conforme se incrementa la cantidad de siluetas.

● RESULTADOS

Car

Voxel



Conclusiones y trabajo futuro

Conclusiones

- El desempeño depende de la representación; no es una propiedad estática del método.
- Voxel falla por sustracción destructiva; Mesh, por adición indeseada de masa.
- El tipo de silueta y su combinación determinan el éxito o el colapso.

Limitaciones

Reproducibilidad sin seeds; límites geométricos en voxel y topológicos en mesh.

Trabajo futuro

Control de determinismo, regularización adaptativa y normalización de siluetas.